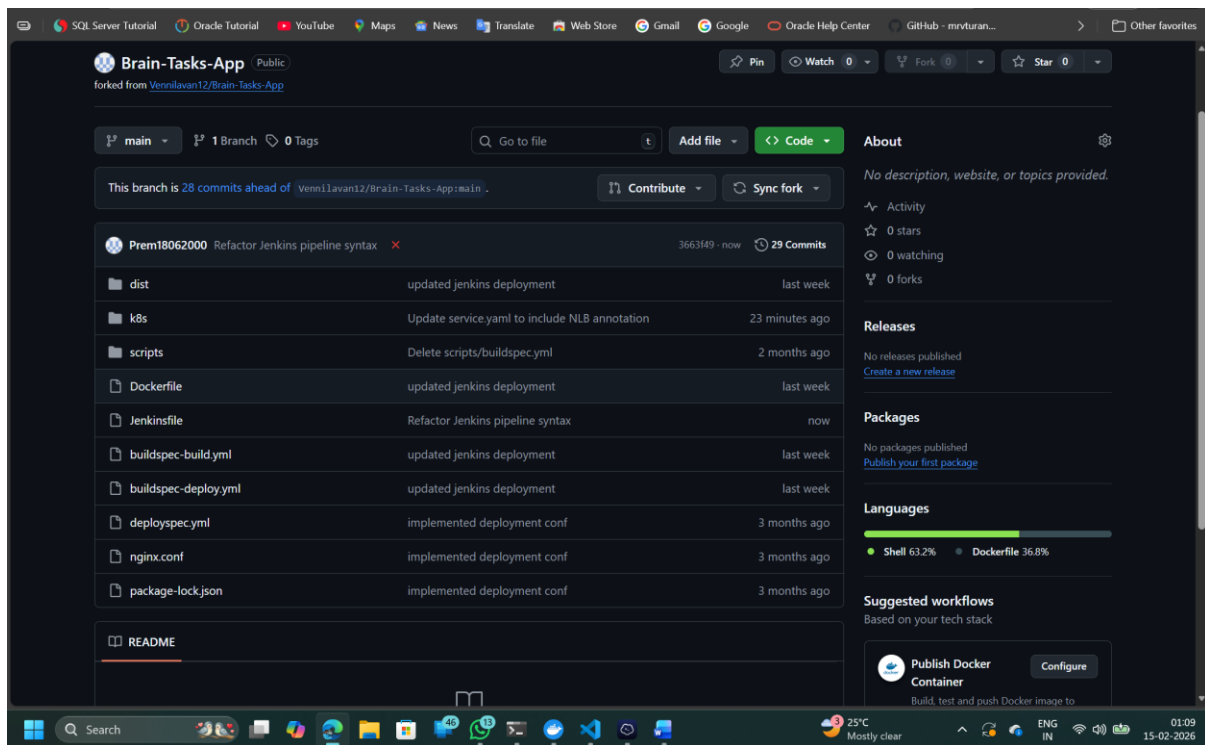


Brain Tasks App DevOps Documentation

This document provides end-to-end DevOps guidance for the **Brain Tasks App**, a web application built with the Vite toolchain. It is aimed at **DevOps engineers** and covers building, containerization, CI/CD, deployment to Kubernetes, and monitoring. All sensitive values (AWS IDs, secrets, etc.) are redacted or masked.

Overview

The Brain Tasks App is a **single-page web application** built with **Vite** as the build tool. In development, source files (HTML/CSS/JS or a framework like React/Vue) are served by Vite's dev server. For production, running `npm run build` (with Vite) generates optimized static files in the `dist/` directory[1]. These assets include an `index.html` and hashed CSS/JS under `dist/assets/` (content-hashed for cache-busting)[2]. The built files can be served by any static file server. In this project, the static site is ultimately served by **NGINX** in a Docker container (see [Docker Build](#)).



Build Process

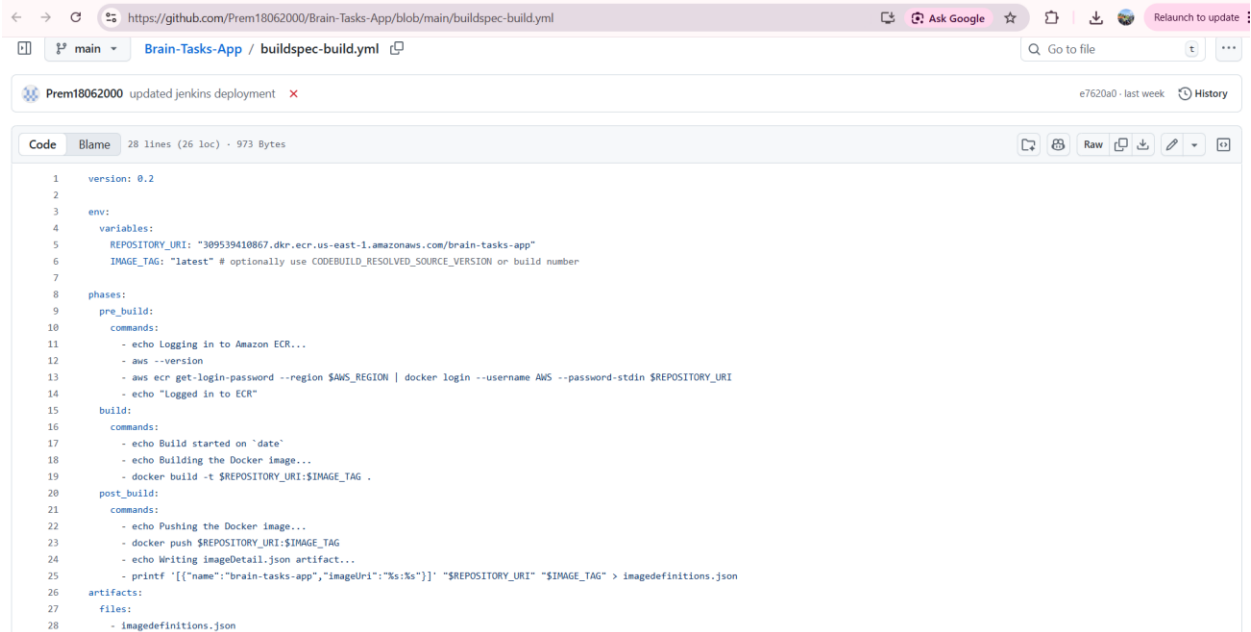
To build the app for production, use the NPM script defined in `package.json`. Typically this is:

```
npm install      # install dependencies
npm run build    # runs `vite build`
```

Running `npm run build` invokes Vite's build process and outputs the production-ready site to `dist/[1]`. The output folder contains `index.html` and static assets (JS/CSS/images) with hashed filenames for cache control. For example, Vite's static asset handling ensures referenced assets get hashed names in the production build[2]. You can verify the build locally with:

```
npm run preview    # serves the built files locally (not for production)
```

This spins up a lightweight server (by default on `http://localhost:4173`) to check that the `dist/` output works as expected[3].

A screenshot of a web browser displaying a GitHub repository page for 'Brain-Tasks-App'. The browser's address bar shows the URL 'https://github.com/Prem18062000/Brain-Tasks-App/blob/main/buildspec-build.yml'. The page title is 'Brain-Tasks-App / buildspec-build.yml'. Below the title, there's a notification 'Prem18062000 updated jenkins deployment'. The main content area shows the 'Code' tab for the file 'buildspec-build.yml', which is 28 lines long, 26 loc, and 973 Bytes. The code is a YAML file defining a build process. It includes sections for 'version' (0.2), 'env' (with 'variables' for 'REPOSITORY_URI' and 'IMAGE_TAG'), 'phases' (with 'pre_build' and 'build' commands), 'post_build' commands, and 'artifacts' (listing 'imagedefinitions.json'). The 'pre_build' phase includes commands for logging in to Amazon ECR. The 'build' phase includes commands for building the Docker image. The 'post_build' phase includes commands for pushing the Docker image and writing an artifact. The 'artifacts' section lists the 'imagedefinitions.json' file.

```
1 version: 0.2
2
3 env:
4   variables:
5     REPOSITORY_URI: "309539410867.dkr.ecr.us-east-1.amazonaws.com/brain-tasks-app"
6     IMAGE_TAG: "latest" # optionally use CODEBUILD_RESOLVED_SOURCE_VERSION or build number
7
8 phases:
9   pre_build:
10    commands:
11      - echo Logging in to Amazon ECR...
12      - aws --version
13      - aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $REPOSITORY_URI
14      - echo "Logged in to ECR"
15   build:
16    commands:
17      - echo Build started on `date`
18      - echo Building the Docker image...
19      - docker build -t $REPOSITORY_URI:$IMAGE_TAG .
20   post_build:
21    commands:
22      - echo Pushing the Docker image...
23      - docker push $REPOSITORY_URI:$IMAGE_TAG
24      - echo Writing imageDetail.json artifact...
25      - printf '{"name":"brain-tasks-app","imageUri":"%s:%s"}' "$REPOSITORY_URI" "$IMAGE_TAG" > imagedefinitions.json
26 artifacts:
27   files:
28     - imagedefinitions.json
```

Dockerization

For deployment, the app is **containerized** using a multi-stage Docker build. The Dockerfile typically has two stages:

1. **Builder stage (Node):** Uses a Node image to install dependencies and run the Vite build.
2. **Runner stage (NGINX):** Uses an NGINX image to serve the built files.

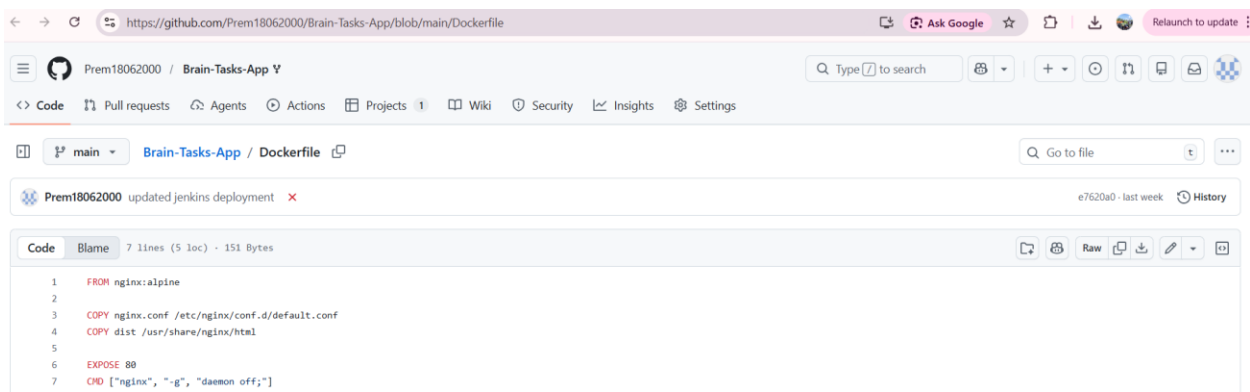
An example Dockerfile pattern is shown in [Naimul Haque's blog][39], which is very similar to this project's setup:

```
FROM node:alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
```

```
FROM nginx:alpine
```

```
COPY --from=builder /app/dist /usr/share/nginx/html
RUN rm /etc/nginx/conf.d/default.conf
COPY ./nginx.conf /etc/nginx/conf.d
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

In this pattern, the first stage installs Node modules and builds the app. The second stage copies the `dist/` folder into NGINX's web root (`/usr/share/nginx/html`)[4]. An `nginx.conf` is provided (in the repo) to configure routing. Typically, the NGINX config listens on port 80 (mapped to the container), serves static files, and includes a `try_files $uri /index.html;` rule so that all routes fall back to `index.html` (necessary for client-side routing in SPAs)[5]. Finally, the Dockerfile exposes port 80. The result is a lightweight web server container that delivers the static app.



CI/CD Pipeline

Continuous integration and deployment are automated. This project supports both **Jenkins**-based and **AWS CodePipeline**-based workflows, as hinted by the presence of a `Jenkinsfile` and `AWS buildspec` files.

- **AWS CodeBuild / CodePipeline:** The repo includes `buildspec-build.yml` and `buildspec-deploy.yml`. In AWS CodePipeline, the **Build** stage can use CodeBuild with `buildspec-build.yml`, and the **Deploy** stage can use CodeBuild (or CodeDeploy) with `buildspec-deploy.yml`[6]. A *buildspec* is a YAML file that specifies build commands, environment variables, and artifacts for CodeBuild[7]. For example, in a similar AWS pipeline setup the repo shows:

```
├─ buildspec-build.yml    # CodeBuild (Build stage)
├─ buildspec-deploy.yml   # CodeBuild (Deploy stage)
```

which separates the build and deploy logic[6]. In the buildspec, typical phases include installing dependencies, running tests, building the Docker image, and pushing to a registry. The deploy buildspec might run `kubectl` commands to apply manifests to EKS.

- **Jenkins:** The `Jenkinsfile` defines a declarative pipeline with stages. In a common pattern, Jenkins checks out the code, installs Node modules, builds the app (with `npm run build`), then builds a Docker image, and pushes it to **Amazon ECR**. Terence Wong's example notes: *"build a Docker image with a Jenkinsfile workflow and publish the image to Amazon Elastic Container Registry (ECR). Jenkins will trigger a deployment to Amazon EKS."*[8]. In other words, Jenkins automates the same steps: build the Vite app, Dockerize it, push to ECR, then deploy to EKS.

Example (simplified) Jenkins pipeline steps: 1. **Build Stage:** `npm ci && npm run build`, then `docker build -t ${ECR_URI} .` 2. **Push Stage:** Tag and push the image to ECR. 3. **Deploy Stage:** Use `kubectl` (with `kubeconfig`) or a deploy script (see `scripts/deploy.sh`) to apply Kubernetes manifests (changing the image tag, etc). The blog notes that AWS credentials (access key/secret) are downloaded (e.g. in `rootkey.csv`) and stored in Jenkins credentials for authentication[9].

Sensitive keys from `rootkey.csv` should be **secured**. As the blog advises, create an IAM user and download its Access Key ID and Secret Access Key, then use those values in Jenkins credentials (never hardcode them)[9]. The `deploy.sh` script in `scripts/` likely runs `kubectl apply -f k8s/app/deployment.yaml` and `service.yaml` to update the running application.

- **Secrets & Masking:** No AWS account IDs or secret keys are shown here. In the CI/CD configs, placeholders or environment variables should be used. For example, the Docker image URI might look like `123456789012.dkr.ecr.us-east-1.amazonaws.com/brain-tasks-app:latest`, but "123456789012" (account ID) should be masked in documentation. Always store credentials in a secrets manager or CI credentials store rather than in source files.

Jenkins

+ New Item

Build History

Build Queue

No builds in the queue.

Build Executor Status

0/2

All

+

S	W	Name	Last Success	Last Failure	Last Duration
✓	☁	brain-tasks-app-pipeline	1 min 3 sec #30	23 min #27	13 sec

Icon: S M L

REST API Jenkins 2.528.2

Jenkins

brain-tasks-app-pipeline

Status

Changes

Build Now

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

brain-tasks-app-pipeline

Permalinks

- Last build (#30), 49 sec ago
- Last stable build (#30), 49 sec ago
- Last successful build (#30), 49 sec ago
- Last failed build (#27), 23 min ago
- Last unsuccessful build (#27), 23 min ago
- Last completed build (#30), 49 sec ago

Builds

Filter

Today

✓ #30 7:39 PM

✓ #29 7:36 PM

✓ #28 7:24 PM

✗ #27 7:16 PM

✗ #26 7:14 PM

✗ #25 7:04 PM

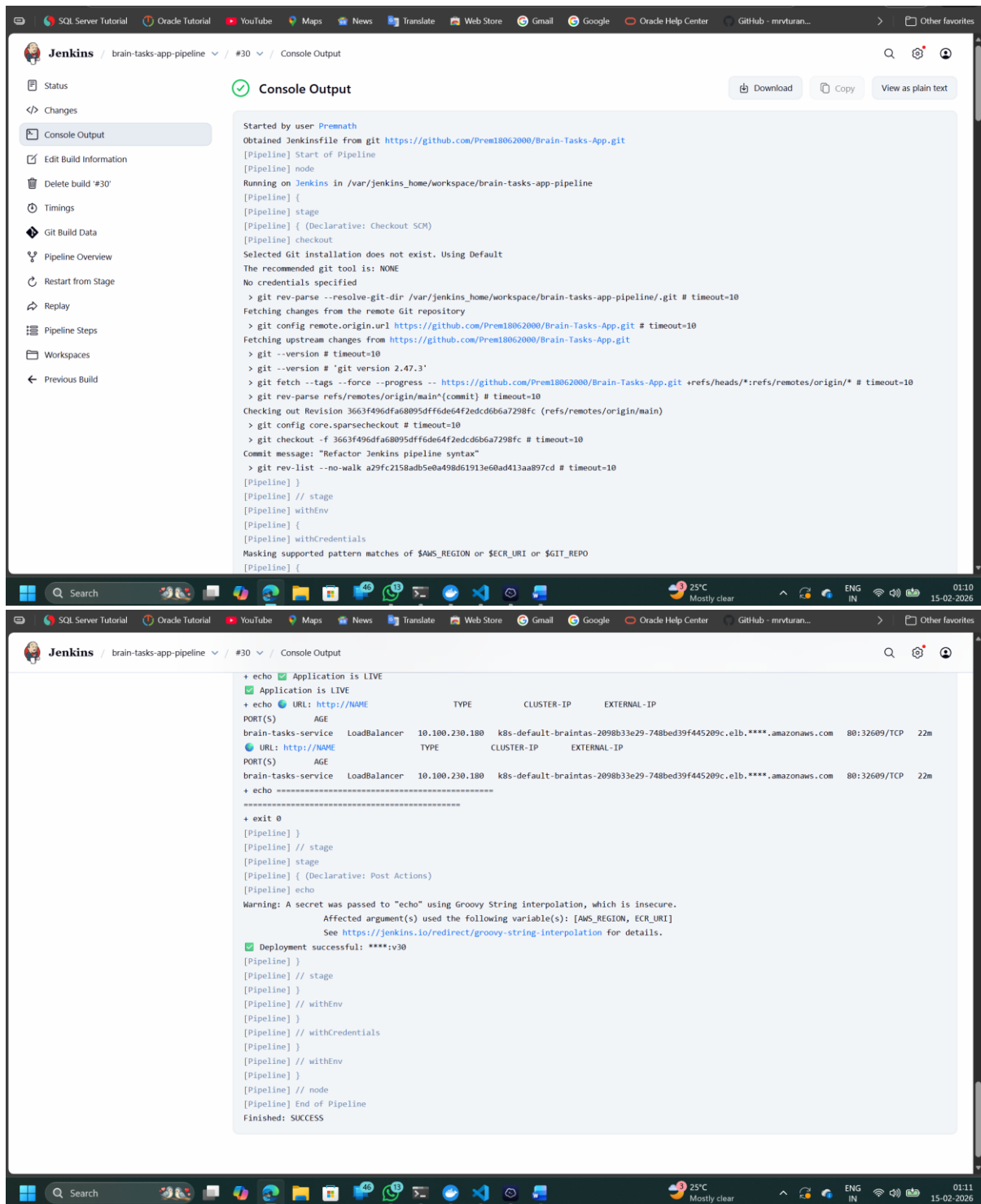
✗ #24 7:04 PM

✓ #23 6:58 PM

✓ #22 6:49 PM

✓ #21 6:18 PM

✗ #20 5:01 PM



Kubernetes Deployment

The application is deployed to **Kubernetes (Amazon EKS)**. Under k8s/app, there are two key manifests:

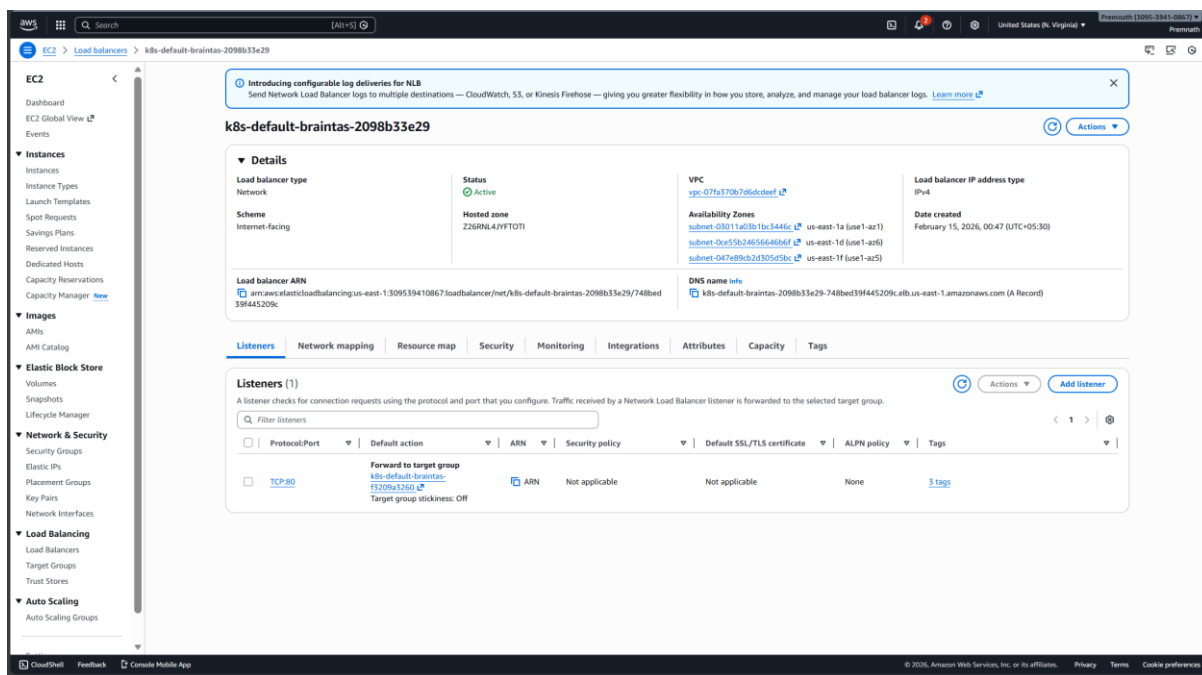
- **deployment.yaml:** Defines a Kubernetes Deployment object (apiVersion: apps/v1). This specifies the container image (from ECR), number of replicas, and pod template. A Deployment “manages a set of Pods to run an application workload” in a declarative way[10]. For example, if it sets replicas: 3, Kubernetes ensures three pods are running the given container image. Updating the image or other spec fields triggers a rolling update.
- **service.yaml:** Defines a Service that exposes the app. In AWS, this is typically type LoadBalancer, which provisions an ELB to expose the app externally. As one pipeline description confirms, the app is “Exposed via LoadBalancer (port 3000)”[11]. Thus the Service maps port 3000 of the pods to a public endpoint. After deployment, navigating to the LoadBalancer’s DNS will serve the app on port 3000 (or a custom domain via Ingress/Route53).

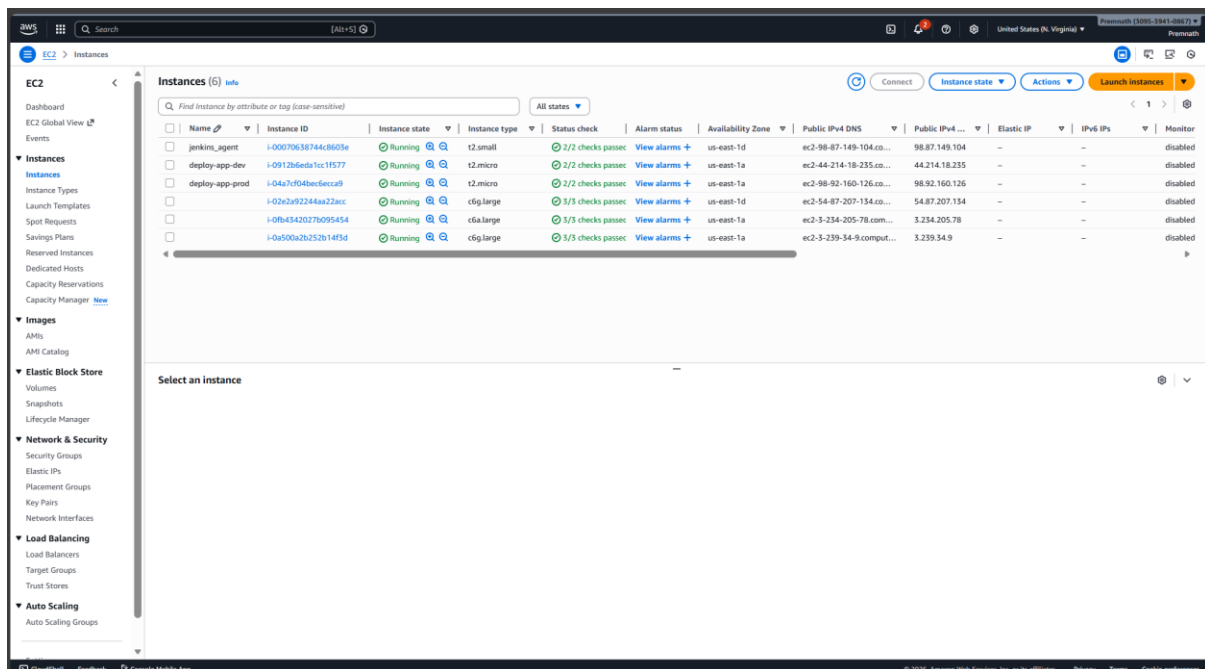
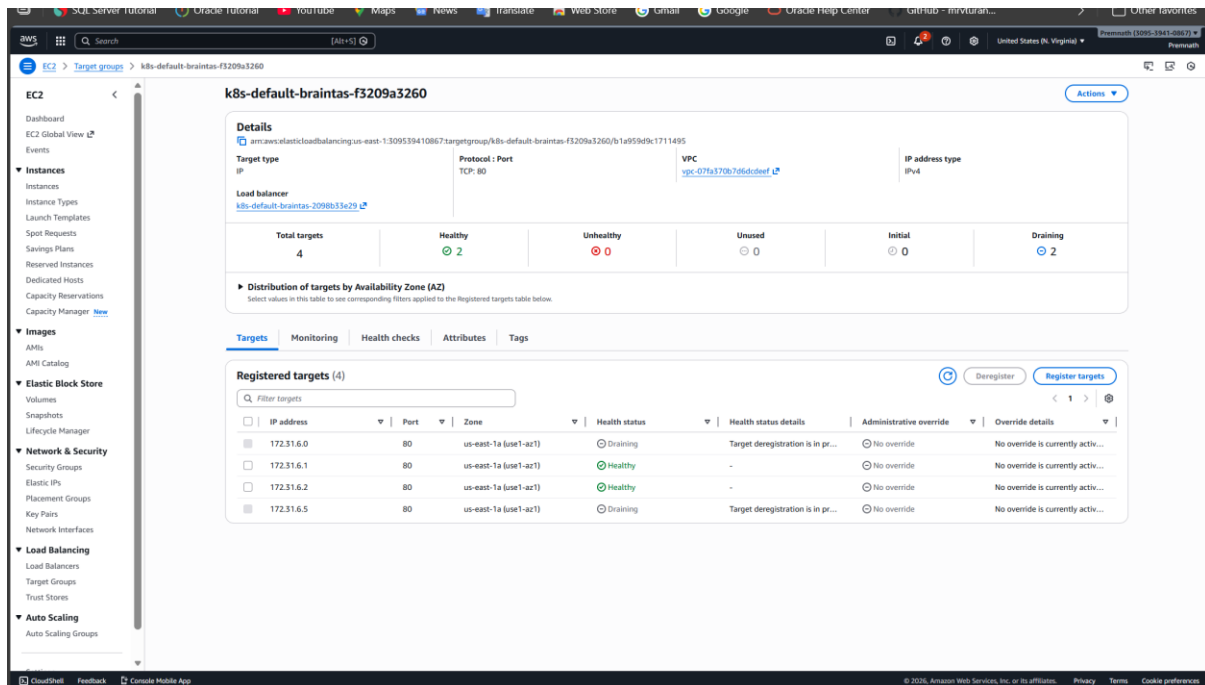
To deploy, the CI/CD pipeline (whether Jenkins or CodeBuild) applies these manifests. For example:

```
kubectl apply -f k8s/app/deployment.yaml
kubectl apply -f k8s/app/service.yaml
```

Kubernetes will then pull the image from ECR and start the pods. Because the Service is a LoadBalancer, you can access the app at the allocated public IP (e.g. `http://<ELB-DNS>:3000`).

Additional Kubernetes notes: - Ensure kubectl is configured with the EKS cluster’s context (via AWS IAM auth). The Jenkins/CodeBuild agent must have permissions (eks:DescribeCluster, IAM role or assume-role for EKS access). - Rollbacks can be managed by rolling back the Deployment if needed (Kubernetes tracks revisions).





Monitoring and Service Mesh

This project uses **Kuma** as a service mesh and observability solution. The k8s/monitoring folder likely contains manifests to install Kuma's control plane in Kubernetes. Typically these include: - kuma-deployment.yaml: Deploys the Kuma control plane. - kuma-service.yaml: Exposes the Kuma control plane (ClusterIP or

LoadBalancer). - `kuma-pvc.yaml`: A PersistentVolumeClaim for the control plane's storage needs (e.g. for Prometheus data or control plane state).

Kuma provides out-of-the-box observability integrations. Running `kumactl install observability` (or applying the monitoring manifests) will create a mesh-observability namespace with Prometheus, Jaeger, Loki, and Grafana set up to work with Kuma^[12]. In fact, Kuma's docs state: *"This will create a namespace mesh-observability with prometheus, jaeger, loki and grafana installed and setup to work with Kuma."*^[12]. These tools collect metrics, logs, and traces from the mesh.

By integrating Kuma, the app gains: - **mTLS and traffic metrics**: Kuma proxies auto-expose service metrics to Prometheus. - **Built-in dashboards**: Grafana can be pointed at the Kuma/Prometheus data. - **Tracing**: Jaeger ingests request traces across services in the mesh.

DevOps engineers should verify that the Kuma control plane is running (`kubectl -n kuma-system get pods` etc) and that the monitoring stack is in mesh-observability. Grafana can be accessed (usually via port-forward or a service) to visualize metrics. The `kuma-pvc.yaml` ensures that Prometheus metrics (or Grafana dashboards) persist across restarts.

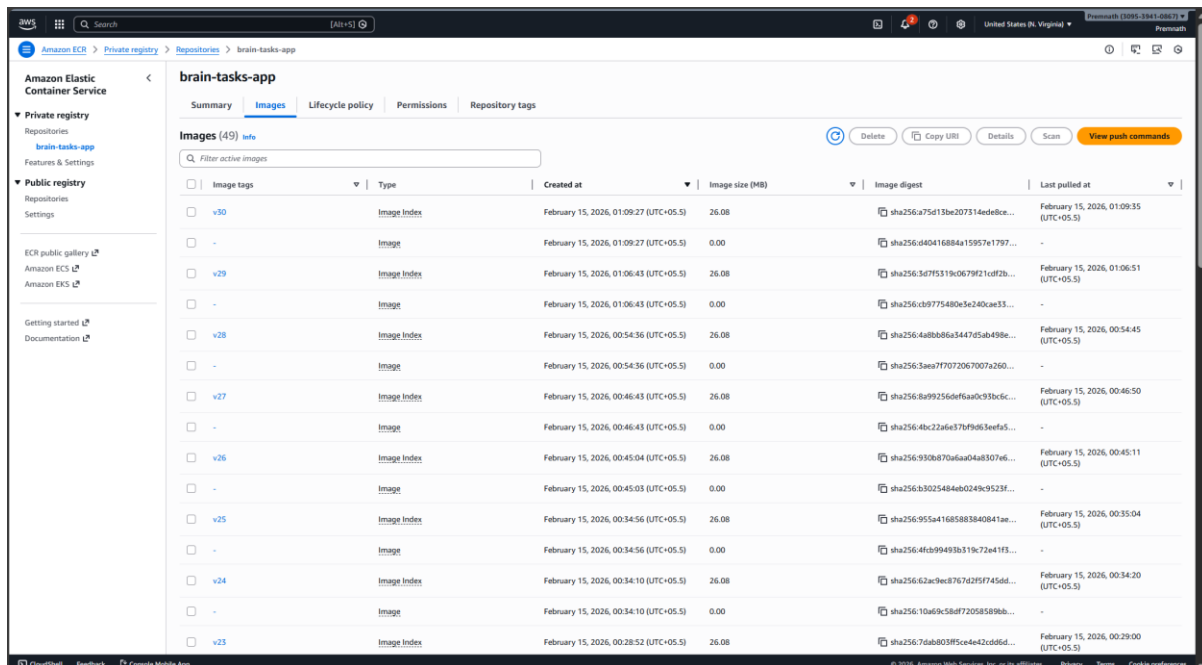
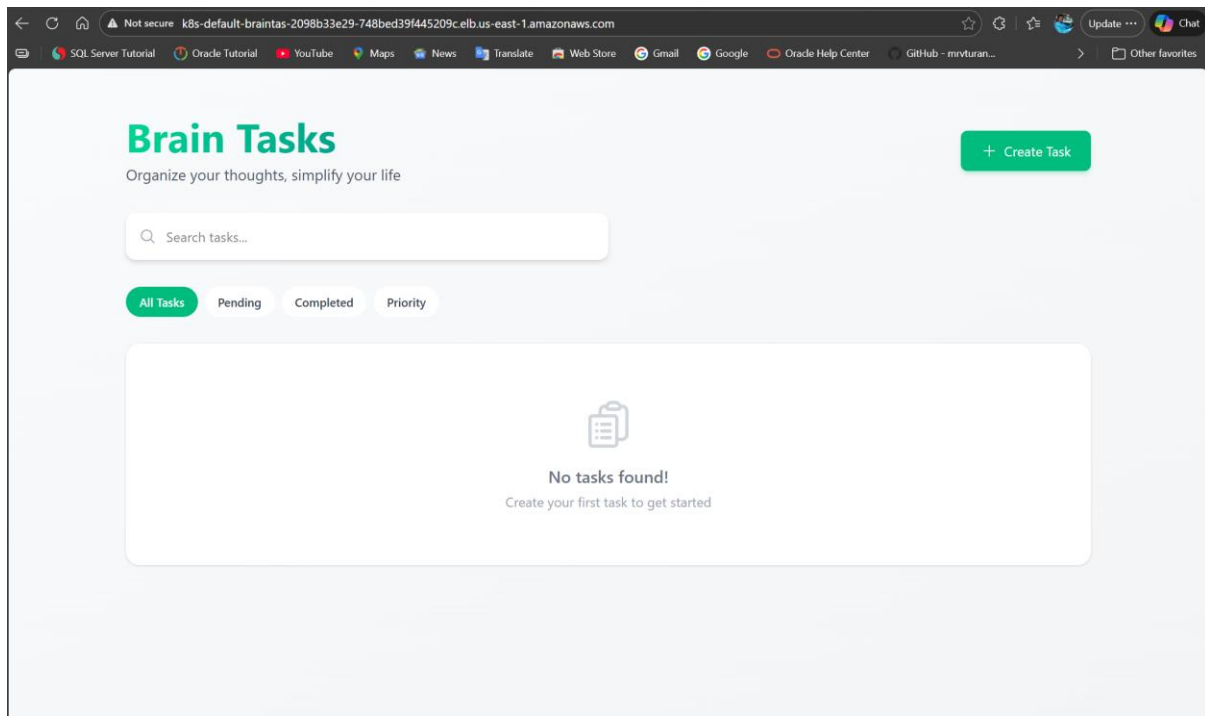
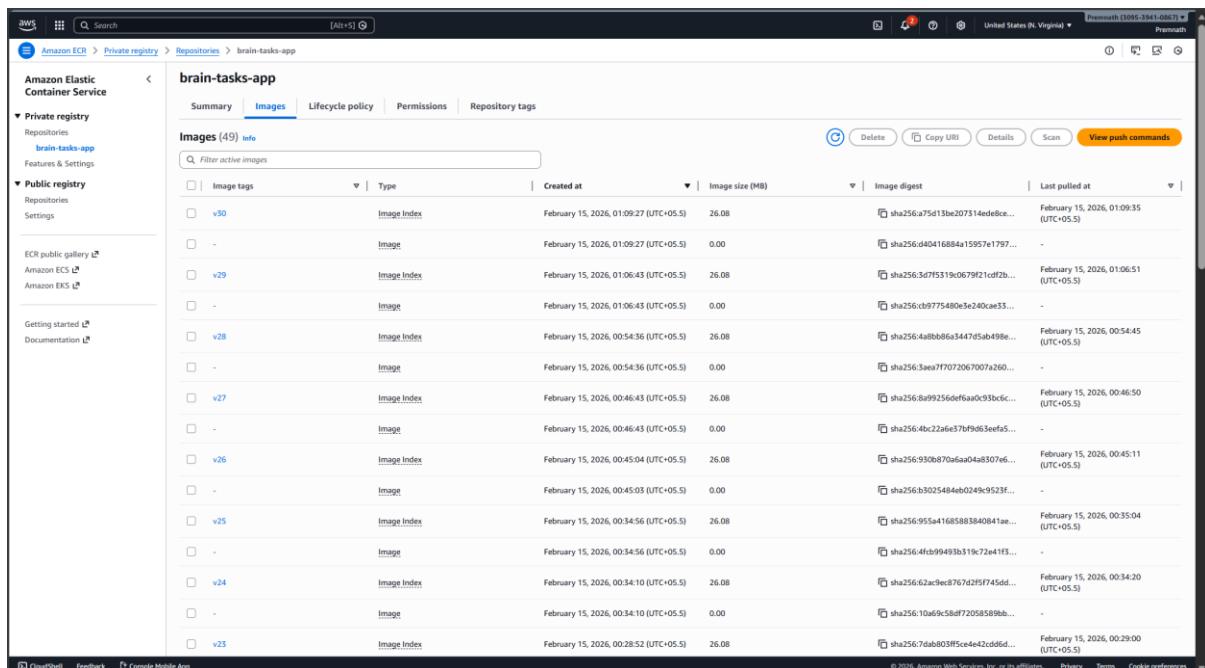


Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
v30	Image Index	February 15, 2026, 01:09:27 (UTC+05:5)	26.08	sha256:a75d13bc207314ede3ce...	February 15, 2026, 01:09:35 (UTC+05:5)
-	Image	February 15, 2026, 01:09:27 (UTC+05:5)	0.00	sha256:640416884a15957e1797...	-
v29	Image Index	February 15, 2026, 01:06:43 (UTC+05:5)	26.08	sha256:3d7f5319c0679f21cdf3b...	February 15, 2026, 01:06:51 (UTC+05:5)
-	Image	February 15, 2026, 01:06:43 (UTC+05:5)	0.00	sha256:cb9775480e3c240cae53...	-
v28	Image Index	February 15, 2026, 00:54:36 (UTC+05:5)	26.08	sha256:4a8b866a3447e5ab498e...	February 15, 2026, 00:54:45 (UTC+05:5)
-	Image	February 15, 2026, 00:54:36 (UTC+05:5)	0.00	sha256:3aea7f7072067007a260...	-
v27	Image Index	February 15, 2026, 00:46:43 (UTC+05:5)	26.08	sha256:8a99256d6f6a0b093bc6c...	February 15, 2026, 00:46:50 (UTC+05:5)
-	Image	February 15, 2026, 00:46:43 (UTC+05:5)	0.00	sha256:4bc22a6e17b9f063eeffa5...	-
v26	Image Index	February 15, 2026, 00:45:04 (UTC+05:5)	26.08	sha256:930b870afaa04a8307e6...	February 15, 2026, 00:45:11 (UTC+05:5)
-	Image	February 15, 2026, 00:45:03 (UTC+05:5)	0.00	sha256:b3025484eb0249c9523f...	-
v25	Image Index	February 15, 2026, 00:34:56 (UTC+05:5)	26.08	sha256:955a41685883840841ae...	February 15, 2026, 00:35:04 (UTC+05:5)
-	Image	February 15, 2026, 00:34:56 (UTC+05:5)	0.00	sha256:4fcd99493b319c72e41f3...	-
v24	Image Index	February 15, 2026, 00:34:10 (UTC+05:5)	26.08	sha256:62ac9ec876742f5745dd...	February 15, 2026, 00:34:20 (UTC+05:5)
-	Image	February 15, 2026, 00:34:10 (UTC+05:5)	0.00	sha256:10a69c58df72058589bb...	-
v23	Image Index	February 15, 2026, 00:28:52 (UTC+05:5)	26.08	sha256:7dab8039f5c4e4e2cd96d...	February 15, 2026, 00:29:00 (UTC+05:5)



Environment and Security

- AWS Credentials:** The file `rootkey.csv` contains AWS access keys (likely for an admin IAM user). **Never commit these.** Instead, configure CI/CD to use IAM roles or securely injected credentials. For example, Jenkins or CodeBuild should use IAM roles with limited scope.

- **Masking:** In examples and docs, we use placeholders like `AWS_ACCOUNT_ID` or `REDACTED`. Ensure real values (account IDs, region, domain names) are not exposed.
- **Nginx Config (`nginx.conf`):** This config handles request routing. It should be reviewed to ensure correct ports and proxies. In production, consider enabling SSL/TLS (or use an AWS ALB with SSL certificate) since default is HTTP on port 80/3000.
- **IAM and Least Privilege:** The CI/CD runner (Jenkins agent or CodeBuild role) needs permissions to ECR, EKS, and CloudWatch Logs. For example, push to ECR requires `ecr:BatchCheckLayerAvailability`, `ecr:PutImage`, etc. EKS deploy requires `eks:DescribeCluster`, `eks:AccessKubernetesApi`, etc. Avoid using root-level keys; use scoped IAM users/roles.

Summary of Steps

1. **Local Build:** Run `npm ci && npm run build` to generate `dist/`.
2. **Docker Build:** Build the Docker image (multi-stage) so that `dist/` is served by NGINX.
3. **Push Image:** Push the image to AWS ECR (as set in the Docker build step).
4. **Deployment:** CI/CD applies the Kubernetes manifests (`deployment.yaml`, `service.yaml`) to the EKS cluster. The Deployment uses the new image tag, and the Service exposes it on port 3000[11].
5. **Monitoring:** Ensure Kuma and its observability stack are deployed (`k8s/monitoring`) so that Prometheus/Grafana collect metrics from the mesh[12].
6. **Validation:** Confirm that the LoadBalancer is up (check `kubectl get svc`) and that accessing the endpoint serves the app. Verify logs/metrics in CloudWatch/Grafana.

All instructions assume DevOps knowledge of tools like `kubectl`, AWS CLI, IAM, and Jenkins/CodePipeline. For more details on any technology used here, see the official docs: Kubernetes Deployments/Services[10], AWS CodeBuild buildspecs[7], and Kuma observability guide[12].

[1] [3] Deploying a Static Site | Vite

<https://vite.dev/guide/static-deploy>

[2] Static Asset Handling | Vite

<https://vite.dev/guide/assets>

[4] [5] Dockerfile for Deploying Vite React SPAs with Nginx

<https://naimulhaque.com/snippets/dockerfile-for-deploying-vite-react-sp-as-with-nginx>

[6] #kubernetes #aws #eks #devops #cicd #codepipeline #codebuild #externaldns #ingress #cloud | Aslam Parvaiz

https://www.linkedin.com/posts/aslam-chandio_kubernetes-aws-eks-activity-7361379973951111169-RICp

[7] Build specification reference for CodeBuild - AWS CodeBuild

<https://docs.aws.amazon.com/codebuild/latest/userguide/build-spec-ref.html>

[8] [9] Deploying To Amazon EKS With Docker And Jenkins | Octopus blog

<https://octopus.com/blog/jenkins-eks-ecr-deployment>

[10] Deployments | Kubernetes

<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

[11] Secure Kubernetes Access to S3 with OIDC and Secrets Manager | Vandana Pandit posted on the topic | LinkedIn

https://www.linkedin.com/posts/vandanapndt_devops-kubernetes-eks-activity-7425592347054383104-CXKd

[12] Observability | Kuma

<https://kuma.io/docs/2.13.x/explore/observability/>